

1장 요구사항 확인



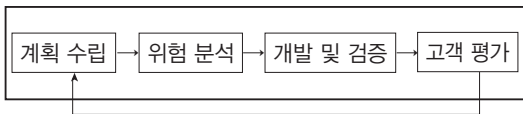
340002 필기 25.8, 25.5, 25.2, 24.5, 23.7, 23.5, 23.2, 22.7, 21.8, 21.3, 20.9, 20.8, 20.6



001 나선형 모형

(B)

- 나선형 모형(Spiral Model, 점진적 모형)은 나선을 따라 돌듯이 여러 번의 소프트웨어 개발 과정을 거쳐 점진적으로 완벽한 최종 소프트웨어를 개발하는 모형이다.
- 보헴(Boehm)이 제안하였다.
- 4가지 주요 활동



340003 필기 24.7, 24.2, 21.8, 20.9, 20.8, 20.6



002 폭포수 모형

(B)

- 폭포수 모형(Waterfall Model)은 이전 단계로 돌아갈 수 없다는 전제하에 각 단계를 확실히 매듭짓고 그 결과를 철저하게 검토하여 승인 과정을 거친 후에 다음 단계를 진행하는 개발 방법론이다.
- 가장 오래되고 가장 폭넓게 사용된 전통적인 소프트웨어 생명 주기 모형이다.
- 고전적 생명 주기 모형이라고도 한다.

공급해요 [7]

Q 실기 책에 왜 필기의 기출 년월이 표시되어 있나요?

A 정보처리기사 시험은 필기와 실기가 시험 범위가 같습니다. 동일한 내용이 객관식으로 필기시험에 나올 수도 있고, 단답형이나 서술식으로 실기시험에 나올 수도 있습니다. 공부하다 보면 알겠지만 필기시험과 실기시험에 중복해서 나온 내용이 많습니다. 자격 시험에는 나온 문제가 또 나온다는 걸 명심하세요.

340005

20.7, 필기 25.8, 23.7, 22.4, 22.3, 21.8, 21.5, 20.8



003 애자일 모형

(A)

- 애자일(Agile)은 '민첩한', '기민한'이라는 의미로, 고객의 요구사항 변화에 유연하게 대응할 수 있도록 일정한 주기를 반복하면서 개발하는 모형이다.
- 어느 특정 개발 방법론이 아니라 좋은 것을 빠르고 낭비 없게 만들기 위해 고객과의 소통에 초점을 맞춘 방법론을 통칭한다.
- 대표적인 개발 모형
 - 스크럼(Scrum)
 - XP(eXtreme Programming)
 - 칸반(Kanban)
 - Lean
 - 기능 중심 개발(FDD; Feature Driven Development)

340006

필기 25.8, 24.7, 23.2, 22.3, 21.3, 20.8



004 애자일 개발 4가지 핵심 가치

(B)

- 프로세스와 도구보다는 개인과 상호작용에 더 가치를 둔다.
- 방대한 문서보다는 실행되는 SW에 더 가치를 둔다.
- 계약 협상보다는 고객과 협업에 더 가치를 둔다.
- 계획을 따르기 보다는 변화에 반응하는 것에 더 가치를 둔다.

340010

필기 25.8, 25.2, 24.5, 24.2, 23.7, 23.5, 23.2, 22.7, 22.4, 20.9, 20.6



005 XP



- XP(eXtreme Programming)는 수시로 발생하는 고객의 요구사항에 유연하게 대응하기 위해 고객의 참여와 개발 과정의 반복을 극대화하여 개발 생산성을 향상시키는 방법이다.
- 짧고 반복적인 개발 주기, 단순한 설계, 고객의 적극적인 참여를 통해 소프트웨어를 빠르게 개발하는 것을 목적으로 한다.
- XP의 5가지 핵심 가치
 - 의사소통(Communication)
 - 단순성(Simplicity)
 - 용기(Courage)
 - 존중(Respect)
 - 피드백(Feedback)

440009

20.10, 필기 24.7, 24.5, 22.4, 20.9



006 XP의 주요 실천 방법(Practice)



필기 20.9

Pair Programming(짝 프로그래밍)

다른 사람과 함께 프로그래밍을 수행함으로써 개발에 대한 책임을 공동으로 나눠 갖는 환경을 조성한다.

필기 20.9

Collective Ownership(공동 코드 소유)

개발 코드에 대한 권한과 책임을 공동으로 소유한다.

Test-Driven Development(테스트 주도 개발)

- 개발자가 실제 코드를 작성하기 전에 테스트 케이스를 먼저 작성하므로 자신이 무엇을 해야할지를 정확히 파악한다.
- 테스트가 지속적으로 진행될 수 있도록 자동화된 테스트 도구(구조, 프레임워크)를 사용한다.

Whole Team(전체 팀)

개발에 참여하는 모든 구성원(고객 포함)들은 각자 자신의 역할이 있고 그 역할에 대한 책임을 가져야 한다.

필기 20.9

Continuous Integration(계속적인 통합)

모듈 단위로 나눠서 개발된 코드들은 하나의 작업이 마무리될 때마다 지속적으로 통합된다.

20.10, 필기 24.7, 24.5, 22.4

Refactoring(리팩토링)

- 프로그램 기능의 변경 없이 시스템을 재구성한다.
- 목적 : 프로그램을 쉽게 이해하고 쉽게 수정하여 빠르게 개발할 수 있도록 하기 위함임

Small Releases(소규모 릴리즈)

릴리즈 기간을 짧게 반복함으로써 고객의 요구 변화에 신속히 대응할 수 있다.

340016

21.4, 필기 22.4



007 기능 요구사항



- 기능 요구사항(Functional Requirements)은 시스템이 무엇을 하는지, 어떤 기능을 하는지 등의 기능이나 수행과 관련된 요구사항이다.
- 시스템의 입력이나 출력으로 무엇이 포함되어야 하는지에 대한 사항
- 시스템이 어떤 데이터를 저장하거나 연산을 수행해야 하는지에 대한 사항
- 시스템이 반드시 수행해야 하는 기능
- 사용자가 시스템을 통해 제공받기를 원하는 기능

340017

21.4, 필기 25.5, 24.5, 23.2, 22.4, 21.8



008 비기능 요구사항



- 비기능 요구사항(Non-functional Requirements)은 품질이나 제약사항과 관련된 요구사항이다.
- 시스템 장비 구성 요구사항
- 성능 요구사항
- 인터페이스 요구사항
- 데이터를 구축하기 위해 필요한 요구사항
- 테스트 요구사항
- 보안 요구사항
- 품질 요구사항 : 가용성, 정합성, 상호 호환성, 대응성, 이식성, 확장성, 보안성 등



009 요구사항 분석

C

- 요구사항 분석(Requirement Analysis)은 소프트웨어 개발의 실제적인 첫 단계로, 개발 대상에 대한 사용자의 요구사항을 이해하고 문서화하는 활동을 의미한다.
- 사용자 요구의 타당성을 조사하고 비용과 일정에 대한 제약을 설정한다.
- 사용자의 요구를 정확하게 추출하여 목표를 정한다.



010 자료 흐름도의 구성 요소

A

기호	의미
필기 25.5, 25.2, 24.5, 23.2, 22.7, ... 프로세스(Process)	자료를 변환시키는 시스템의 한 부분(처리 과정)을 나타내며 처리, 기능, 변환, 버블이라고도함
필기 25.5, 25.2, 24.5, 23.2, 22.7, ... 자료 흐름(Data Flow)	자료의 이동(흐름)이나 연관관계를 나타냄
필기 25.5, 25.2, 24.5, 23.2, 22.7, ... 자료 저장소(Data Store)	시스템에서의 자료 저장소(파일, 데이터베이스)를 나타냄
필기 25.5, 24.5, 23.2, 22.7, 22.3, ... 단말(Terminator)	시스템과 교신하는 외부 개체로, 입력 데이터가 만들어지고 출력 데이터를 받음



011 HIPO

B

- HIPO(Hierarchy Input Process Output)는 시스템의 분석 및 설계, 또는 문서화에 사용되는 기법으로, 시스템 실행 과정인 입력·처리·출력의 기능을 표현한 것이다.
- 하향식 소프트웨어 개발을 위한 문서화 도구이다.
- 시스템의 기능을 여러 개의 고위 모듈로 분할하여 이들 간의 인터페이스를 계층 구조로 표현한 것을 HIPO Chart라고 한다.



012 연관 관계

A

- 연관(Association) 관계는 2개 이상의 사물이 서로 관련되어 있는 관계이다.
- 사물 사이를 실선으로 연결하여 표현한다.
- 방향성은 화살표로 표현한다.
- 양방향 관계의 경우 화살표를 생략하고 실선으로만 연결한다.
- 다중도를 선 위에 표기한다.

예제 사람이 집을 소유하는 관계이다. 사람은 자기가 소유하고 있는 집에 대해 알고 있지만 집은 누구에 의해 자신 이 소유되고 있는지 모른다는 의미이다.



013 집합 관계

A

- 집합(Aggregation) 관계는 하나의 사물이 다른 사물에 포함되어 있는 관계이다.
- 포함하는 쪽(전체, Whole)과 포함되는 쪽(부분, Part)은 서로 독립적이다.
- 포함되는 쪽(부분, Part)에서 포함하는 쪽(전체, Whole)으로 속이 빈 마름모를 연결하여 표현한다.

예제 프린터는 컴퓨터에 연결해서 사용할 수 있으며, 다른 컴퓨터에 연결해서 사용할 수도 있다.



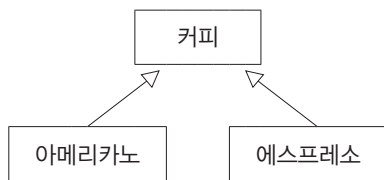


014 일반화 관계

(A)

- 일반화(Generalization) 관계는 하나의 사물이 다른 사물에 비해 더 일반적이거나 구체적인 관계이다.
- 보다 일반적인 개념을 상위(부모), 보다 구체적인 개념을 하위(자식)라고 부른다.
- 구체적(하위)인 사물에서 일반적(상위)인 사물 쪽으로 속이 빈 화살표를 연결하여 표현한다.

예제 아메리카노와 에스프레소는 커피이다. 다시 말하면, 커피에는 아메리카노와 에스프레소가 있다.



015 의존 관계

(A)

- 의존(Dependency) 관계는 연관 관계와 같이 사물 사이에 서로 연관은 있으나 필요에 의해 서로에게 영향을 주는 짧은 시간 동안만 연관을 유지하는 관계이다.
- 하나의 사물과 다른 사물이 소유 관계는 아니지만 사물의 변화가 다른 사물에도 영향을 미치는 관계이다.
- 일반적으로 한 클래스가 다른 클래스를 오퍼레이션의 매개 변수로 사용하는 경우에 나타나는 관계이다.
- 영향을 받는 사물(이용자)이 영향을 주는 사물(제공자) 쪽으로 점선 화살표를 연결하여 표현한다.

예제 등급이 높으면 할인율을 적용하고, 등급이 낮으면 할인율을 적용하지 않는다.



016 구조적 다이어그램의 종류

(B)

필기 25.8, 23.2, 21.5, 21.3, 20.6

클래스 다이어그램(Class Diagram)

클래스와 클래스가 가지는 속성, 클래스 사이의 관계를 표현한다.

필기 20.6

객체 다이어그램(Object Diagram)

- 클래스에 속한 사물(객체)들, 즉 인스턴스(Instance)를 특정 시점의 객체와 객체 사이의 관계로 표현한다.
- 럼바우(Rumbaugh) 객체지향 분석 기법에서 객체 모델링에 활용된다.

필기 23.5, 22.3, 20.6

컴포넌트 다이어그램(Component Diagram)

- 실제 구현 모듈인 컴포넌트 간의 관계나 컴포넌트 간의 인터페이스를 표현한다.
- 구현 단계에서 사용된다.

필기 22.3

배치 다이어그램(Deployment Diagram)

- 결과물, 프로세스, 컴포넌트 등 물리적 요소들의 위치를 표현한다.
- 구현 단계에서 사용된다.

복합체 구조 다이어그램(Composite Structure Diagram)

클래스나 컴포넌트가 복합 구조를 갖는 경우 그 내부 구조를 표현한다.

필기 22.3

패키지 다이어그램(Package Diagram)

유스케이스나 클래스 등의 모델 요소들을 그룹화한 패키지들의 관계를 표현한다.



017 행위 다이어그램의 종류



필기 23.5, 20.8

유스케이스 다이어그램(Use Case Diagram)

- 사용자의 요구를 분석하는 것으로, 기능 모델링 작업에 사용한다.
- 사용자(Actor)와 사용 사례(Use Case)로 구성된다.

필기 25.8, 23.5, 23.2, 22.3, 21.5, 21.3, 20.8

순차 다이어그램(Sequence Diagram)

상호작용하는 시스템이나 객체들이 주고받는 메시지를 표현한다.

커뮤니케이션 다이어그램(Communication Diagram)

동작에 참여하는 객체들이 주고받는 메시지와 객체들 간의 연관 관계를 표현한다.

필기 25.8, 23.5, 21.3, 20.9

상태 다이어그램(State Diagram)

- 하나의 객체가 자신이 속한 클래스의 상태 변화 혹은 다른 객체와의 상호작용에 따라 상태가 어떻게 변화하는지를 표현한다.
- 럼바우(Rumbaugh) 객체지향 분석 기법에서 동적 모델링에 활용된다.

필기 25.8, 23.2, 21.5, 20.8

활동 다이어그램(Activity Diagram)

시스템이 어떤 기능을 수행하는지 객체의 처리 로직이나 조건에 따른 처리의 흐름을 순서에 따라 표현한다.

상호작용 개요 다이어그램(Interaction Overview Diagram)

상호작용 다이어그램 간의 제어 흐름을 표현한다.

타이밍 다이어그램(Timing Diagram)

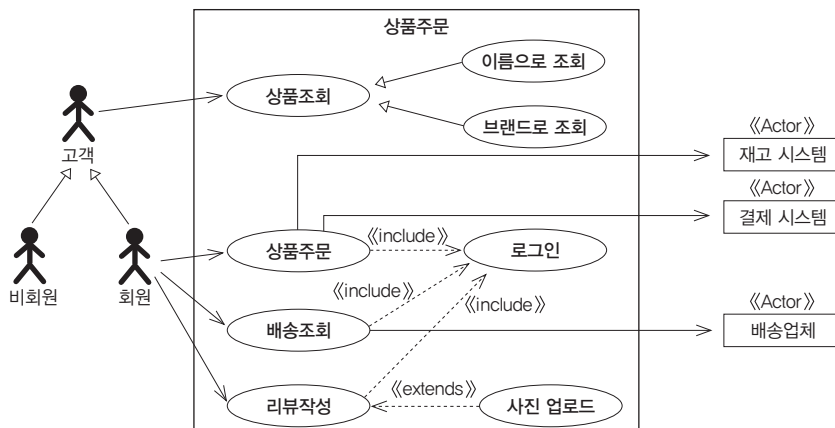
객체 상태 변화와 시간 제약을 명시적으로 표현한다.



- 유스케이스(Use Case) 다이어그램은 사용자와 다른 외부 시스템들이 개발될 시스템을 이용해 수행할 수 있는 기능을 사용자의 관점에서 표현한 것이다.
- 유스케이스 다이어그램의 구성 요소

구성 요소	표현 방법	내용
시스템(System) / 시스템 범위(System Scope)	상품주문	시스템 내부의 유스케이스들을 사각형으로 묶어 시스템의 범위를 표현한 것
필기 21.5 액터(Actor)	• 주액터 고객 • 부액터 <<Actor>> 재고 시스템	• 시스템과 상호작용을 하는 모든 외부 요소 • 주로 사람이나 외부 시스템을 의미함 • 주액터 : 시스템을 사용함으로써 이득을 얻는 대상으로, 주로 사람이 해당됨 • 부액터 : 주액터의 목적 달성을 위해 시스템에 서비스를 제공하는 외부 시스템으로, 조직이나 기관 등이 될 수 있음
유스케이스(Use Case)	상품조회	사용자가 보는 관점에서 시스템이 액터에게 제공하는 서비스나 기능을 표현한 것
관계(Relationship)	• 포함 <<include>> • 확장 <<extends>> • 일반화 	• 유스케이스 다이어그램에서 관계는 액터와 유스케이스, 유스케이스와 유스케이스 사이에서 나타날 수 있음 • 유스케이스에서 나타날 수 있는 관계 : 포함(Include) 관계, 확장(Extends) 관계, 일반화(Generalization) 관계

예제 다음은 회원과 비회원으로 구분되는 회원이 상품주문 과정에서 수행할 수 있는 기능을 표현한 유스케이스 다이어그램이다.





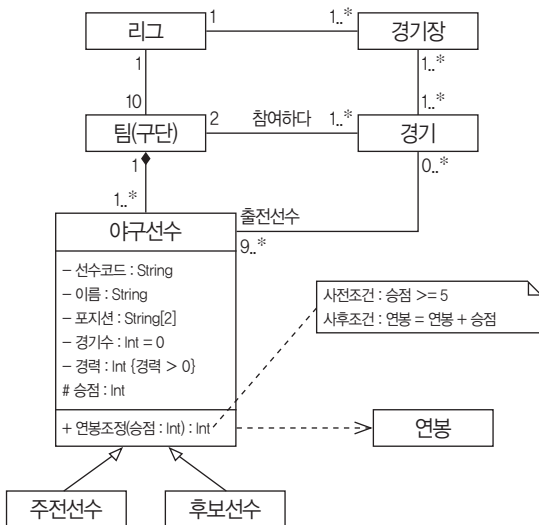
019 클래스 다이어그램



- 클래스(Class) 다이어그램은 클래스와 클래스가 가지는 속성, 클래스 사이의 관계를 표현한 것이다.
- 클래스 다이어그램의 구성 요소

구성 요소	표현 방법	내용
21.10, 필기 21.8 클래스(Class)	클래스명 속성1 속성2 오퍼레이션1 오퍼레이션2	• 각각의 객체들이 갖는 속성과 오퍼레이션(동작)을 표현한 것 • 일반적으로 3개의 구획(Compartment)으로 나뉘 클래스의 이름, 속성, 오퍼레이션을 표기함 • 속성(Attribute) : 클래스의 상태나 정보를 표현함 • 오퍼레이션(Operation) : 클래스가 수행할 수 있는 동작으로, 함수(메소드, Method)라고도 함
제약조건		• 속성에 입력될 값에 대한 제약조건이나 오퍼레이션 수행 전후에 지정해야 할 조건이 있다면 이를 적음 • 클래스 안에 제약조건을 기술할 때는 중괄호 { }를 이용함
관계(Relationships)		• 관계는 클래스와 클래스 사이의 연관성을 표현함 • 클래스 다이어그램에 표현하는 관계에는 연관 관계, 집합 관계, 포함 관계, 일반화 관계, 의존 관계가 있음

예제 다음은 프로야구 리그에 필요한 정보의 일부를 표현한 클래스 다이어그램이다.





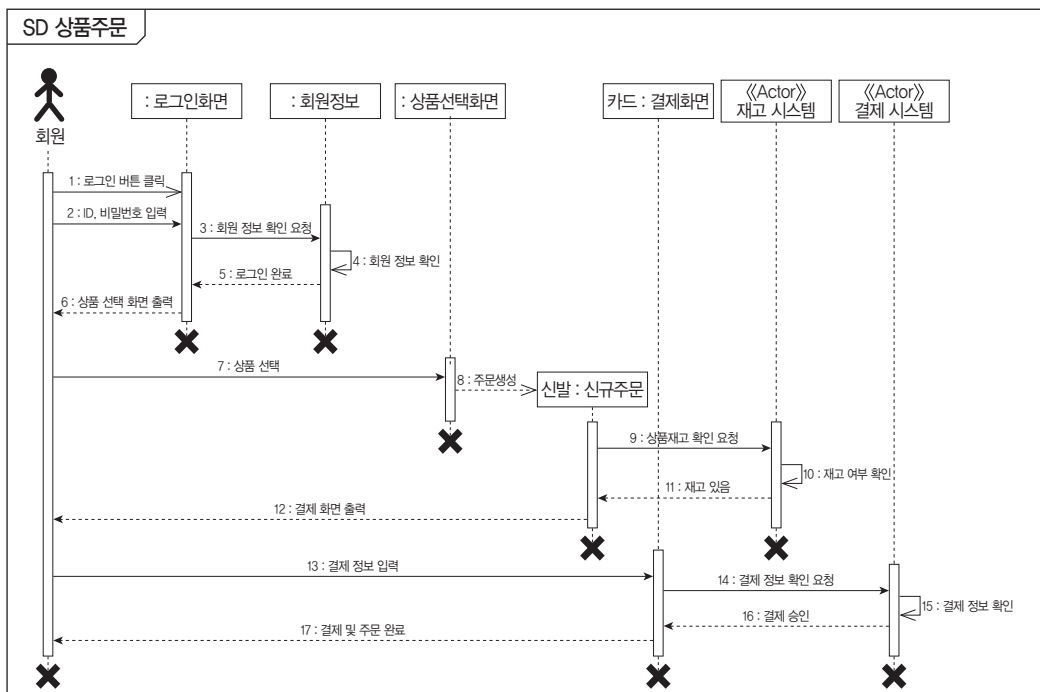
020 순차 다이어그램

(B)

- 순차(Sequence) 다이어그램은 시스템이나 객체들이 메시지를 주고받으며 상호작용하는 과정을 그림으로 표현한 것이다.
- 순차 다이어그램의 구성 요소

구성 요소	표현 방법	의미
액터(Actor)	 회원	시스템으로부터 서비스를 요청하는 외부 요소로, 사람이나 외부 시스템을 의미함
객체(Object)	: 로그인화면	메시지를 주고받는 주체
필기 22.7, 20.8 생명선(Lifeline)		• 객체가 메모리에 존재하는 기간으로, 객체 아래쪽에 점선을 그어 표현함 • 객체 소멸(✕)이 표시된 기간까지 존재함
필기 22.7, 20.8 실행 상자 (Active Box, 활성 상자)		객체가 메시지를 주고받으며 구동되고 있음을 표현함
필기 22.7, 20.8 메시지(Message)	1: 로그인 버튼 클릭 →	객체가 상호 작용을 위해 주고받는 메시지
객체 소멸	✕	해당 객체가 더 이상 메모리에 존재하지 않음을 표현한 것
프레임(Frame)	SD 상품주문	다이어그램의 전체 또는 일부를 묶어 표현한 것

예제 다음은 회원의 상품 주문 과정에 재고 시스템과 결제 시스템이 관계된 상호 작용 과정을 표현한 순차 다이어그램이다.





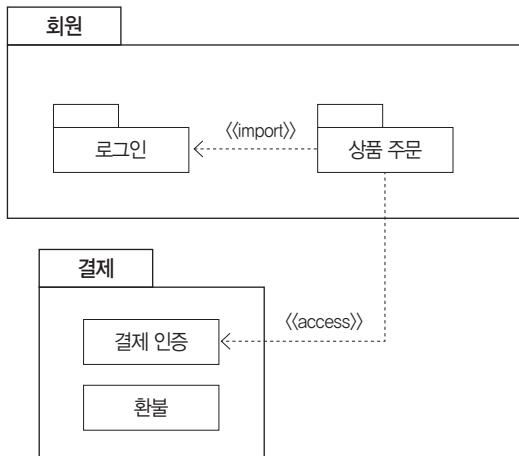
021 패키지 다이어그램



- 패키지(Package) 다이어그램은 유스케이스나 클래스 등의 요소들을 그룹화한 패키지 간의 의존 관계를 표현한 것이다.
- 패키지 다이어그램의 구성 요소

구성 요소	표현 방법	의미
패키지(Package)		- 객체들을 그룹화한 것 - 단순 표기법 : 패키지 안에 패키지 이름만 표현 - 확장 표기법 : 패키지 안에 요소까지 표현
객체(Object)		유스케이스, 클래스, 인터페이스, 테이블 등 패키지에 포함될 수 있는 다양한 요소들
의존 관계 (Dependency)		- 패키지와 패키지, 패키지와 객체 간을 점선 화살표로 연결하여 표현함 - 스테레오 타입을 이용해 의존 관계를 구체적으로 표현할 수 있음 - 의존 관계의 표현 형태는 사용자가 임의로 작성할 수 있으며, 대표적으로 import와 access가 사용됨 《import》: 패키지에 포함된 객체들을 직접 가져와서 이용하는 관계 《access》: 인터페이스를 통해 패키지 내의 객체에 접근하여 이용하는 관계

예제 다음은 회원이 상품 주문 시 사용하는 〈회원〉, 〈로그인〉, 〈상품 주문〉, 〈결제〉 패키지들 간의 의존 관계를 표현한 패키지 다이어그램이다.



340051

필기 21.3, 20.9



022 컴포넌트 기반 방법론

C

- 컴포넌트 기반 방법론(CBD; Component Based Design)은 기존의 시스템이나 소프트웨어를 구성하는 컴포넌트를 조합하여 하나의 새로운 애플리케이션을 만드는 방법론이다.
- 컴포넌트의 재사용(Reusability)이 가능하여 시간과 노력을 절감할 수 있다.
- 컴포넌트 기반 방법론의 개발 절차



340054

필기 25.5, 25.2, 23.7, 23.5, 21.3, 20.9, 20.8, 20.6



023 CASE

B

- CASE(Computer Aided Software Engineering)는 소프트웨어 개발 과정에서 사용되는 요구 분석, 설계, 구현, 검사 및 디버깅 과정 전체 또는 일부를 컴퓨터와 전용 소프트웨어 도구를 사용하여 자동화하는 것이다.
- CASE의 주요 기능
 - 소프트웨어 생명 주기 전 단계의 연결
 - 다양한 소프트웨어 개발 모형 지원
 - 그래픽 지원

340057

20.5, 필기 23.2, 22.7, 22.4, 22.3, 21.8, 21.3, 20.6



024 LOC 기법

A

- LOC(source Line Of Code, 원시 코드 라인 수) 기법은 소프트웨어 각 기능의 원시 코드 라인 수의 비관치, 낙관치, 기대치를 측정하여 예측치를 구하고 이를 이용하여 비용을 산정하는 기법이다.
- 산정 공식
 - 노력(인월) = 개발 기간 × 투입 인원

$$= LOC / 1인당 월평균 생산 코드 라인 수$$
 - 개발 비용

$$= 노력(인월) \times 단위 비용(1인당 월평균 인건비)$$
 - 개발 기간 = 노력(인월) / 투입 인원
 - 생산성 = LOC / 노력(인월)

340058

필기 21.5, 20.9



025 수학적 산정 기법

C

- 수학적 산정 기법은 상향식 비용 산정 기법으로, 경험적 추정 모형, 실험적 추정 모형이라고도 한다.
- 수학적 산정 기법은 개발 비용 산정의 자동화를 목표로 한다.
- 주요 수학적 산정 기법
 - COCOMO 모형
 - Putnam 모형
 - 기능 점수(FP) 모형

440047

필기 25.2, 22.7, 22.4



026 COCOMO 모형

C

- COCOMO(Constructive COst Model) 모형은 원시 프로그램의 규모인 LOC(원시 코드 라인 수)에 의한 비용 산정 기법이다.
- 개발할 소프트웨어의 규모(LOC)를 예측한 후 이를 소프트웨어 종류에 따라 다르게 책정되는 비용 산정 방정식에 대입하여 비용을 산정한다.
- 비용 산정 결과는 프로젝트를 완성하는 데 필요한 노력(Man-Month)으로 나타난다.
- 보헴(Boehm)이 제안하였다.

340060

필기 25.5, 23.5, 23.2, 22.4, 21.8, 21.5, 21.3, 20.8, 20.6



027 COCOMO의 소프트웨어 개발 유형

B

필기 23.5, 23.2, 22.4, 21.8, 21.5, 21.3, 20.8, 20.6

조직형(Organic Mode)

- 기관 내부에서 개발된 중·소 규모의 소프트웨어이다.
- 일괄 자료 처리나 과학기술 계산용, 비즈니스 자료 처리용 등의 5만(50KDSI) 라인 이하의 소프트웨어를 개발하는 유형이다.
- 사무 처리용, 업무용, 과학용 응용 소프트웨어 개발에 적합하다.

반분리형(Semi-Detached Mode)

- 조직형과 내장형의 중간형 소프트웨어이다.
- 트랜잭션 처리 시스템이나 운영체제, 데이터베이스 관리 시스템 등의 30만(300KDSI) 라인 이하의 소프트웨어를 개발하는 유형이다.
- 컴파일러, 인터프리터와 같은 유틸리티 개발에 적합하다.

내장형(Embedded Mode)

- 초대형 규모의 소프트웨어이다.
- 트랜잭션 처리 시스템이나 운영체제 등의 30만(300KDSI) 라인 이상의 소프트웨어를 개발하는 유형이다.
- 신호기 제어 시스템, 미사일 유도 시스템, 실시간 처리 시스템 등의 시스템 프로그램 개발에 적합하다.



028 Putnam 모형

ⓒ

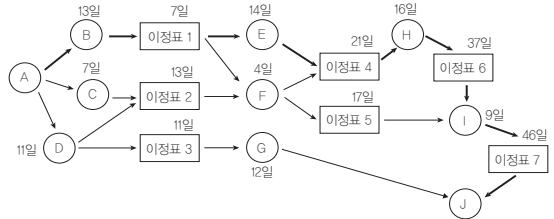
- Putnam 모형은 소프트웨어 생명 주기의 전 과정 동안에 사용될 노력의 분포를 예상하는 모형이다.
- 푸트남(Putnam)이 제안한 것으로, 생명 주기 예측 모형이라고도 한다.
- 시간에 따른 함수로 표현되는 Rayleigh-Norden 곡선의 노력 분포도를 기초로 한다.



029 CPM

Ⓑ

- CPM(Critical Path Method, 임계 경로 기법)은 프로젝트 완성에 필요한 작업을 나열하고 작업에 필요한 소요 기간을 예측하는데 사용하는 기법이다.
- CPM은 노드와 간선으로 구성된 네트워크로 노드는 작업을, 간선은 작업 사이의 전후 의존 관계를 나타낸다.



※ 그림에서 굵은선이 임계 경로, 즉 최장 경로입니다.



030 CMMI

Ⓑ

- CMMI(Capability Maturity Model Integration)는 소프트웨어 개발 조직의 업무 능력 및 조직의 성숙도를 평가하는 모델이다.
- CMMI의 소프트웨어 프로세스 성숙도

단계	특징
초기(Initial)	작업자 능력에 따라 성공 여부 결정
필기 23.2, 20.9, 20.6 관리(Managed)	특정한 프로젝트 내의 프로세스 정의 및 수행
필기 23.2, 20.9, 20.6 정의(Defined)	조직의 표준 프로세스를 활용하여 업무 수행
정량적 관리 (Quantitatively Managed)	프로젝트를 정량적으로 관리 및 통제
필기 20.9, 20.6 최적화 (Optimizing)	프로세스 역량 향상을 위해 지속적인 프로세스 개선



031 SPICE



- SPICE(Software Process Improvement and Capability dEtermination)는 정보 시스템 분야에서 소프트웨어의 품질 및 생산성 향상을 위해 소프트웨어 프로세스를 평가 및 개선하는 국제 표준이다.
- 공식 명칭은 ISO/IEC 15504이다.



032 소프트웨어 개발 프레임워크



- 소프트웨어 개발 프레임워크(Framework)는 소프트웨어 개발에 공통적으로 사용되는 구성 요소와 아키텍처를 일반화하여 손쉽게 구현할 수 있도록 여러 가지 기능들을 제공해주는 반제품 형태의 소프트웨어 시스템이다.
- 선행 사업자의 기술에 의존하지 않는 표준화된 개발 기반으로 인해 사업자 종속성이 해소된다.



033 소프트웨어 개발 프레임워크의 특성



필기 22.7, 21.5, 20.9, 20.6

모듈화(Modularity)

- 프레임워크는 캡슐화를 통해 모듈화를 강화하고 설계 및 구현의 변경에 따른 영향을 최소화함으로써 소프트웨어의 품질을 향상시킨다.
- 프레임워크는 개발 표준에 의한 모듈화로 인해 유지보수가 용이하다.

필기 22.7, 21.5, 20.9, 20.6

재사용성(Reusability)

프레임워크는 재사용 가능한 모듈들을 제공함으로써 예산 절감, 생산성 향상, 품질 보증이 가능하다.

필기 22.7, 21.5

확장성(Extensibility)

프레임워크는 다형성(Polymorphism)을 통한 인터페이스 확장이 가능하여 다양한 형태와 기능을 가진 애플리케이션 개발이 가능하다.

제어의 역흐름(Inversion of Control)

개발자가 관리하고 통제해야 하는 객체들의 제어를 프레임워크에 넘김으로써 생산성을 향상시킨다.